



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería Informática

SmartEye Flow

Analizador de imágenes con flujos
encadenados de inteligencia artificial.



Presentado por Álvaro Pérez Mella
en Universidad de Burgos — 30 de mayo
de 2026

Tutores: José Luis Garrido Labrador y José
Miguel Ramírez Sanz



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. José Luis Garrido Labrador y D. José Miguel Ramírez Sanz, profesores del departamento de Ingeniería Informática, área de Sistemas Informáticos.

Expone:

Que el alumno D. Álvaro Pérez Mella, con DNI 71956263G, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado “SmartEye Flow - Analizador de imágenes con flujos encadenados de inteligencia artificial.”.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 30 de mayo de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. José Luis Garrido Labrador

D. José Miguel Ramírez Sanz

Resumen

Este Trabajo de Fin de Grado presenta el diseño e implementación de una aplicación web orientada al procesamiento de imágenes mediante modelos de inteligencia artificial. El sistema permite a los usuarios autenticados seleccionar flujos de análisis, subir imágenes, ejecutar modelos de visión por computador y obtener como resultado tanto una o varias imágenes procesadas como información estructurada en formato JSON.

El proyecto parte de una primera fase experimental, centrada en validar la integración de modelos como YOLO y MediaPipe, y evoluciona hacia una plataforma más completa basada en Flask, SQLAlchemy y JWT. La aplicación incorpora una arquitectura modular organizada en controladores, servicios, modelos de datos y vistas web, lo que facilita la separación de responsabilidades y la ampliación futura del sistema.

Entre las funcionalidades implementadas destacan la gestión de usuarios y roles, el control de acceso mediante planes y licencias temporales de modelos de IA, la ejecución de *pipelines* formados por varias etapas de procesamiento, la generación de resultados temporales descargables mediante *tokens* y la existencia de una interfaz diferenciada para usuarios y administradores. Además, el proyecto incluye pruebas automatizadas orientadas a verificar el *backend*, análisis estático de calidad mediante SonarCloud y una configuración Docker Compose que permite desplegar la aplicación y la base de datos de forma reproducible.

El resultado es una base funcional y extensible para una plataforma de análisis de imágenes mediante inteligencia artificial, preparada para ser ampliada con nuevos modelos, modos de ejecución y mejoras de despliegue.

Descriptores

Visión por computador, inteligencia artificial, procesamiento de imágenes, aplicación web, Flask, YOLO, MediaPipe, *pipelines*, SQLAlchemy, MariaDB, JWT, autenticación, pruebas automatizadas.

Abstract

This Final Degree Project presents the design and implementation of a web application for image processing using artificial intelligence models. The system allows authenticated users to select analysis workflows, upload images, execute computer vision models and obtain both a processed image or images and structured information in JSON format.

The project began with an experimental stage focused on validating the integration of models such as YOLO and MediaPipe, and later evolved into a more complete platform based on Flask, SQLAlchemy and JWT. The application follows a modular architecture organised into controllers, services, data models and web views, which improves separation of concerns and facilitates future extensions.

The implemented features include user and role management, access control through plans and temporary AI model licences, execution of multi-stage processing pipelines, generation of temporary downloadable results through tokens, and separate interfaces for regular users and administrators. The project also includes automated tests to verify the backend, static code analysis through SonarCloud and a Docker Compose configuration that allows the application and the database to be deployed in a reproducible way.

The result is a functional and extensible foundation for an artificial-intelligence-based image analysis platform, prepared to incorporate new models, execution modes and deployment improvements.

Keywords

Computer vision, artificial intelligence, image processing, web application, Flask, YOLO, MediaPipe, pipelines, SQLAlchemy, MariaDB, JWT, authentication, automated testing.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
Lista de abreviaturas	vii
1. Introducción	1
1.1. Materiales adjuntos	2
2. Objetivos del proyecto	3
2.1. Objetivo general	3
2.2. Objetivos del <i>software</i>	3
2.3. Objetivos técnicos	4
2.4. Objetivos personales	4
3. Conceptos teóricos	5
3.1. Inteligencia artificial, minería de datos y aprendizaje automático	5
3.2. Redes neuronales artificiales	6
3.3. Redes neuronales convolucionales	7
3.4. Procesamiento de imágenes y visión por computador	8
3.5. Arquitecturas de detección de objetos en una etapa	11
4. Técnicas y herramientas	13
4.1. Técnicas	13
4.2. Herramientas	15

5. Aspectos relevantes del desarrollo del proyecto	19
5.1. Validación técnica inicial	20
5.2. Diseño de la aplicación escalable	20
5.3. Consolidación funcional	26
6. Trabajos relacionados	31
6.1. Herramientas y plataformas relacionadas	31
6.2. Comparación con el proyecto desarrollado	32
7. Conclusiones y líneas de trabajo futuras	33
7.1. Conclusiones	33
7.2. Líneas de trabajo futuras	35
Bibliografía	37

Índice de figuras

3.1. Ejemplo de red neuronal artificial.	6
3.2. Ejemplo de capas convolucionales.	7
3.3. Ejemplo de detección de objetos.	9
3.4. Ejemplo de segmentación de imágenes.	11
5.1. Demo inicial con MediaPipe.	21
5.2. Arquitectura modular de la aplicación.	22
5.3. Modelo entidad-relación del sistema.	23
5.4. Flujo de ejecución de pipelines.	25
5.5. Administración de pipelines.	27
5.6. Informe de calidad en SonarCloud.	29

Índice de tablas

3.1. Comparación entre detección de objetos y puntos clave.	10
5.1. Resumen de pruebas automatizadas.	28
6.1. Comparación con trabajos relacionados.	32

Lista de abreviaturas

TFG Trabajo de Fin de Grado.

API *Application Programming Interface.*

CSS *Cascading Style Sheets.*

HTML *HyperText Markup Language.*

JSON *JavaScript Object Notation.*

JWT *JSON Web Token.*

HTTP *HyperText Transfer Protocol.*

ORM *Object-Relational Mapping.*

1. Introducción

En los últimos años, la inteligencia artificial aplicada a la visión por computador ha experimentado un importante crecimiento en tareas como la detección de objetos, la estimación de pose, el reconocimiento de manos o el análisis automático de imágenes. Estas técnicas permiten extraer información útil de una imagen y devolver resultados interpretables, tanto de forma visual como mediante datos estructurados.

Sin embargo, el uso práctico de estos modelos no se limita a ejecutarlos de forma aislada desde un *script*. Para que puedan ser utilizados dentro de una aplicación real, es necesario diseñar un sistema capaz de recibir imágenes, gestionar usuarios y permisos, seleccionar el análisis correspondiente, ejecutar el procesamiento en el servidor y mostrar los resultados de forma clara.

El presente Trabajo de Fin de Grado aborda el diseño e implementación de una plataforma *web* extensible para el procesamiento de imágenes mediante *pipelines* de inteligencia artificial. La finalidad principal no es solo integrar modelos como YOLO o MediaPipe, sino construir una arquitectura modular que permita añadir nuevos modelos, modos de ejecución y flujos de análisis sin modificar de forma significativa el funcionamiento general del sistema.

La aplicación se ha desarrollado con Flask y se organiza en controladores, servicios, modelos de datos y vistas. Esta separación facilita el mantenimiento del código y permite desacoplar la interfaz, la persistencia y la ejecución de los modelos. Además, la ejecución de los *pipelines* se apoya en el patrón factoría, que permite resolver dinámicamente qué modelo y qué modo debe ejecutarse en cada etapa.

Desde el punto de vista funcional, el sistema permite el registro e inicio de sesión de usuarios, la consulta de *pipelines* disponibles, la subida de imágenes, la ejecución del análisis, la visualización de resultados y la descarga temporal

de los archivos generados. También incorpora gestión de planes, alquiler temporal de modelos, historial de ejecuciones y una zona de administración para consultar usuarios, revisar ejecuciones y gestionar *pipelines*.

Finalmente, el proyecto se ha preparado para facilitar su despliegue y validación técnica. La aplicación puede ejecutarse mediante Docker, favoreciendo la reproducibilidad del entorno, y se han desarrollado pruebas unitarias sobre la parte del *backend*, alcanzando una cobertura final del 92,9 %.

1.1. Materiales adjuntos

Junto a esta memoria se entregan los siguientes materiales:

- Anexos técnicos del proyecto.
- Código fuente completo de la aplicación *web*.
- Ficheros de configuración utilizados por los modos de procesamiento.
- *Script* de inicialización de datos para preparar un entorno de demostración.
- Ficheros de despliegue y contenerización mediante Docker.
- Ficheros de configuración para el análisis estático del código con SonarCloud.
- *Tests* unitarios automáticos del *backend*.
- Vídeo de funcionamiento de la aplicación.
- Repositorio del proyecto disponible a través de internet [\[44\]](#).

2. Objetivos del proyecto

A continuación, se detallan los objetivos que han guiado la realización del proyecto, diferenciando entre el objetivo general, los objetivos propios del *software*, los objetivos técnicos y los objetivos personales.

2.1. Objetivo general

El objetivo general del proyecto es desarrollar una plataforma *web* modular y extensible capaz de ejecutar flujos de procesamiento de imágenes mediante inteligencia artificial, devolviendo al usuario resultados visuales y datos estructurados que faciliten la interpretación del análisis realizado.

2.2. Objetivos del *software*

- Permitir que los usuarios se registren, inicien sesión y accedan a un panel propio.
- Ofrecer una interfaz desde la que el usuario pueda consultar los *pipelines* disponibles y seleccionar el análisis que desea realizar.
- Permitir la subida de imágenes y la ejecución del procesamiento correspondiente en el servidor.
- Mostrar al usuario los resultados obtenidos de forma clara, incluyendo la imagen procesada y la información generada por el análisis.
- Incorporar funciones de administración para gestionar usuarios, permisos, servicios disponibles y *pipelines*.

2.3. Objetivos técnicos

- Diseñar una arquitectura modular que separe la interfaz, la lógica de control, la lógica de procesamiento y la persistencia de datos.
- Definir un modelo de datos que permita representar usuarios, permisos, servicios contratados, modelos, modos de ejecución, *pipelines* y resultados.
- Organizar la ejecución de los análisis de forma desacoplada, permitiendo añadir nuevos modelos y modos sin modificar el flujo principal del sistema.
- Gestionar de forma segura las imágenes recibidas y los resultados generados, evitando la exposición directa de rutas internas del servidor.
- Validar el funcionamiento del sistema mediante pruebas automatizadas sobre los componentes principales de la aplicación.

2.4. Objetivos personales

- Aplicar de forma integrada los conocimientos adquiridos durante el grado en desarrollo *web*, bases de datos, ingeniería del *software* e inteligencia artificial.
- Profundizar en el diseño de aplicaciones modulares, mantenibles y preparadas para crecer de forma progresiva.
- Mejorar la capacidad de planificación, organización y documentación de un proyecto completo de desarrollo de *software*.
- Reforzar el uso de pruebas como mecanismo para comprobar el correcto funcionamiento del sistema.
- Acercar el desarrollo del proyecto a una estructura propia de un entorno profesional.

3. Conceptos teóricos

Este capítulo introduce los conceptos teóricos necesarios para comprender la base técnica del trabajo. Se presentan fundamentos de inteligencia artificial, minería de datos, redes neuronales, procesamiento de imágenes y visión por computador. También se explican tareas habituales en el análisis automático de imágenes, como la clasificación, la detección de objetos, la estimación de puntos clave y la segmentación.

3.1. Inteligencia artificial, minería de datos y aprendizaje automático

La inteligencia artificial es el área de la informática que estudia la construcción de sistemas capaces de realizar tareas asociadas a la percepción, el razonamiento, el aprendizaje o la toma de decisiones [47]. Dentro de este campo, el aprendizaje automático se centra en modelos que aprenden patrones a partir de datos, sin depender únicamente de reglas programadas de forma explícita [3].

La minería de datos busca descubrir información útil en conjuntos de datos. Incluye tareas como clasificación, agrupación, detección de anomalías y extracción de patrones frecuentes [22]. En el análisis de imágenes, estas técnicas permiten transformar datos visuales en información interpretable.

La clasificación es una de las tareas más habituales. Consiste en asignar una entrada a una categoría o clase. En imágenes, puede aplicarse a la imagen completa o a regiones concretas, siempre que exista un conjunto de ejemplos etiquetados para entrenar el modelo [55].

El aprendizaje automático puede ser supervisado, no supervisado o por refuerzo. En el aprendizaje supervisado se aprende a partir de ejemplos etiquetados. En el no supervisado se buscan estructuras internas sin etiquetas previas. En el aprendizaje por refuerzo, un agente aprende mediante recompensas o penalizaciones obtenidas al interactuar con un entorno [47].

3.2. Redes neuronales artificiales

Las redes neuronales artificiales son modelos formados por capas de unidades conectadas entre sí. Cada conexión tiene un peso asociado, y cada unidad transforma sus entradas para generar una salida. La combinación de muchas unidades permite aproximar funciones complejas [20].

Una red neuronal suele organizarse en una capa de entrada, una o varias capas ocultas y una capa de salida. La Figura 3.1 muestra una representación simplificada de esta estructura, formada por nodos conectados entre capas.

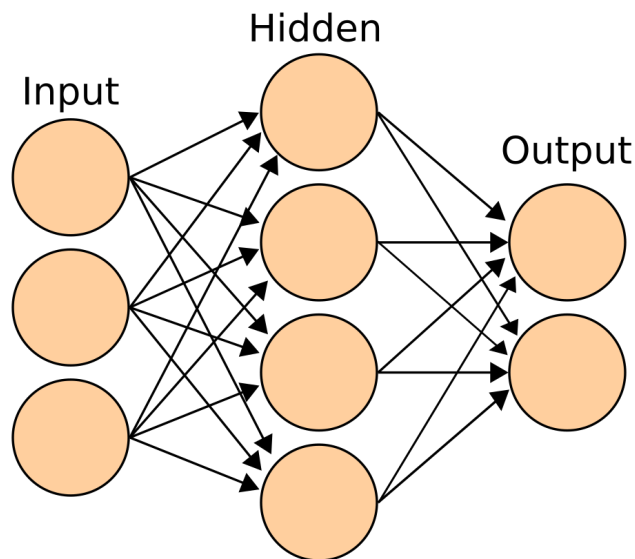


Figura 3.1: Ejemplo de red neuronal artificial con una capa oculta. Fuente: [Cburnett, Wikimedia Commons](#).

Durante el entrenamiento, la red ajusta sus pesos para reducir el error entre la salida estimada y la salida esperada. Este error se mide mediante una función de pérdida, y uno de los mecanismos más utilizados para actualizar los pesos es la retropropagación del error [20].

Una vez entrenada, la red puede utilizarse en fase de inferencia. En esta fase, los pesos no se modifican: el modelo aplica lo aprendido a nuevas entradas. En visión por computador, la entrada suele ser una imagen representada numéricamente, y la salida puede ser una clase, una probabilidad, una caja delimitadora, un conjunto de puntos o una máscara de segmentación.

3.3. Redes neuronales convolucionales

Las redes neuronales convolucionales son redes especialmente adecuadas para trabajar con imágenes. Su operación principal es la convolución, que aplica filtros sobre regiones locales de la imagen para extraer características visuales [32].

En las primeras capas se suelen aprender patrones simples, como bordes, texturas o cambios de intensidad. En capas posteriores, esos patrones se combinan para representar formas y estructuras más complejas [20]. La Figura 3.2 ilustra de forma esquemática esta transformación progresiva.

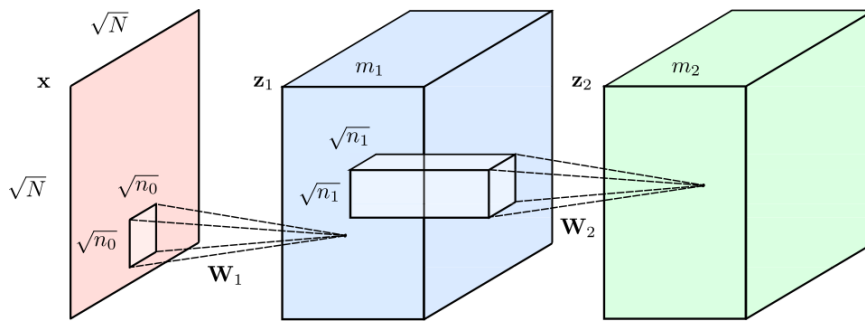


Figura 3.2: Representación simplificada de capas convolucionales en una red neuronal. Fuente: [Renanar2, Wikimedia Commons](#).

Capas habituales en redes convolucionales

Las redes convolucionales combinan distintos tipos de capas. Las más habituales son las siguientes:

- **Capa convolucional.** Aplica filtros sobre la imagen o sobre mapas de características intermedios. Cada filtro aprende a detectar un patrón visual concreto [32].
- **Función de activación.** Introduce no linealidad en la red y permite aprender relaciones más complejas que una transformación lineal [20].

- **Capa de agrupación.** Reduce la resolución espacial de los mapas de características, disminuyendo el coste computacional y conservando la información más relevante [20].
- **Capa totalmente conectada.** Relaciona las salidas de una capa con las unidades de la siguiente. Suele emplearse en fases finales de clasificación.
- **Capa de normalización.** Estabiliza las activaciones durante el entrenamiento y puede acelerar la convergencia del modelo [27].

Estas capas permiten construir modelos capaces de extraer información visual de forma progresiva. Por ello, las redes convolucionales son una base habitual para clasificación de imágenes, detección de objetos, estimación de puntos clave y segmentación.

3.4. Procesamiento de imágenes y visión por computador

Una imagen digital puede representarse como una matriz de píxeles. Cada píxel contiene valores numéricos que describen intensidad o color. En imágenes en escala de grises se utiliza un único canal, mientras que en imágenes a color se emplean varios canales [19].

El procesamiento de imágenes agrupa técnicas orientadas a modificar o preparar una imagen. Incluye operaciones como redimensionado, conversión de color, filtrado, normalización, reducción de ruido o transformación geométrica [19].

La visión por computador busca extraer información útil a partir de imágenes o secuencias de vídeo. Para ello combina procesamiento de imágenes, aprendizaje automático y modelos matemáticos capaces de interpretar contenido visual [51].

Entre sus tareas más habituales se encuentran la clasificación de imágenes, la detección de objetos, la estimación de puntos clave, la segmentación y el seguimiento de elementos en vídeo. Cada una produce un tipo de salida diferente y permite analizar la imagen con distinto nivel de detalle.

Clasificación de imágenes

La clasificación de imágenes consiste en asignar una o varias etiquetas a una imagen completa. El modelo recibe la imagen como entrada y devuelve una clase o una distribución de probabilidades entre varias clases posibles [3].

Esta tarea es útil cuando interesa conocer la categoría global de una imagen. Sin embargo, no proporciona información precisa sobre la posición de los elementos que aparecen en ella.

Detección de objetos

La detección de objetos consiste en localizar entidades dentro de una imagen y asignarles una clase. Su salida habitual incluye una caja delimitadora, una etiqueta y una puntuación de confianza [51].

La Figura 3.3 muestra un ejemplo de detección mediante cajas delimitadoras, donde cada objeto se representa con una región rectangular asociada.

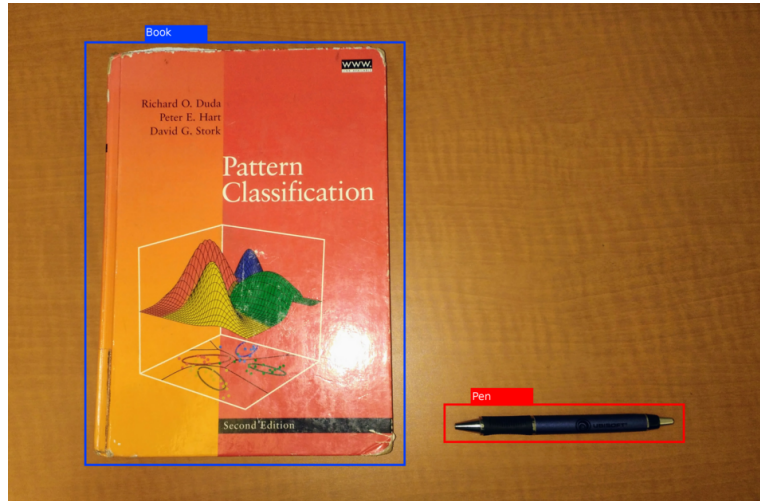


Figura 3.3: Ejemplo de detección de objetos mediante cajas delimitadoras. Fuente: [Hughesperreault](#), [Wikimedia Commons](#).

A diferencia de la clasificación, la detección no solo indica qué aparece en la imagen, sino también dónde se encuentra. Por ello, resulta adecuada cuando una escena puede contener varios elementos de interés.

Estimación de puntos clave

La estimación de puntos clave, también denominados *landmarks*, consiste en localizar posiciones relevantes dentro de una estructura. En lugar de delimitar un objeto completo mediante una caja, identifica puntos concretos, como articulaciones, extremos o referencias anatómicas [51].

Este enfoque permite representar estructuras con mayor detalle. En una mano, los puntos clave pueden corresponder a nudillos y extremos de los dedos. En una pose corporal, pueden representar hombros, codos, muñecas, caderas, rodillas o tobillos.

La Tabla 3.1 recoge las diferencias principales entre la detección de objetos y la estimación de puntos clave.

Aspecto	Detección de objetos	Estimación de puntos clave
Finalidad	Localizar y clasificar entidades visibles en una imagen.	Localizar puntos relevantes de una estructura concreta.
Salida habitual	Caja delimitadora, clase y confianza.	Coordenadas de puntos clave y posibles conexiones entre ellos.
Representación visual	Rectángulos y etiquetas sobre la imagen.	Puntos, esqueleto o malla sobre la imagen.
Nivel de detalle	Describe la posición global del objeto.	Describe la estructura interna del elemento analizado.

Tabla 3.1: Comparación entre detección de objetos y estimación de puntos clave.

Segmentación de imágenes

La segmentación de imágenes consiste en dividir una imagen en regiones significativas. A diferencia de la detección de objetos, que utiliza cajas delimitadoras, la segmentación trabaja a nivel de píxel. Cada píxel puede asignarse a una región, una clase o una instancia concreta [19, 51].

Existen distintos tipos de segmentación. La segmentación semántica asigna una clase a cada píxel sin distinguir objetos diferentes de la misma

clase. La segmentación de instancias diferencia cada objeto individual. La segmentación panóptica combina ambas aproximaciones [33, 29].

La Figura 3.4 muestra un ejemplo visual en el que la imagen queda dividida en varias zonas diferenciadas.

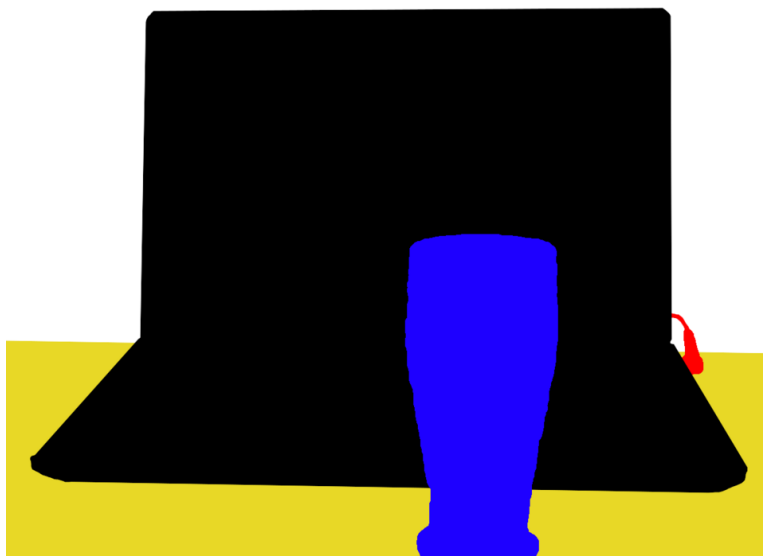


Figura 3.4: Ejemplo de segmentación de imágenes a nivel de regiones. Fuente: [Martin Thoma, Wikimedia Commons](#).

La segmentación proporciona una descripción más precisa que la detección mediante cajas, ya que permite delimitar la forma real de los elementos. Sin embargo, suele requerir modelos más específicos y conjuntos de datos anotados con mayor detalle.

3.5. Arquitecturas de detección de objetos en una etapa

Las arquitecturas de detección de objetos pueden clasificarse, de forma general, en detectores de dos etapas y detectores de una etapa. Los detectores de dos etapas generan primero regiones candidatas y después las clasifican. Los detectores de una etapa realizan la localización y la clasificación de forma directa [45].

La familia de modelos YOLO pertenece al grupo de detectores de una etapa. Su planteamiento original formula la detección como un único problema

de regresión, en el que una red predice cajas delimitadoras y probabilidades de clase en una sola evaluación de la imagen [45].

De forma simplificada, una arquitectura de detección en una etapa incluye tres bloques: extracción de características visuales, combinación de información a distintas escalas y generación de predicciones finales. Estas predicciones contienen coordenadas de cajas, puntuaciones de confianza y clases asociadas.

Tras obtener las predicciones, se aplica un posprocesado para eliminar detecciones redundantes o poco fiables. Una técnica habitual es la supresión de máximos no máximos, que conserva la caja más representativa cuando varias predicciones se solapan sobre el mismo objeto [51].

Este tipo de arquitectura permite resolver detección y clasificación en una misma fase de inferencia. Por ello, es una aproximación habitual en aplicaciones que requieren detección visual eficiente.

4. Técnicas y herramientas

Este capítulo presenta las principales técnicas y herramientas mencionadas en la memoria y en los anexos.

4.1. Técnicas

Gestión y control del trabajo

- **Metodologías ágiles.** Enfoques de gestión basados en desarrollo iterativo, entrega incremental y adaptación continua a los cambios.
- **Scrum.** Marco de trabajo ágil organizado en iteraciones llamadas *sprints*. Cada *sprint* parte de una lista priorizada de trabajo, denominada *Product Backlog*, y finaliza con un incremento revisable del producto.
- **Control de versiones.** Técnica que registra la evolución de un conjunto de archivos y permite recuperar estados anteriores.

Diseño y arquitectura del software

- **Arquitectura modular.** Técnica que divide una aplicación en componentes con responsabilidades diferenciadas, reduciendo el acoplamiento entre partes del sistema.
- **Separación de responsabilidades.** Principio de diseño que asigna cada función del sistema a una capa concreta, evitando concentrar toda la lógica en un único módulo.

- **Modelo-Vista-Controlador.** Patrón arquitectónico que separa una aplicación en tres partes principales: modelo, vista y controlador. Esta separación facilita organizar la lógica de datos, la presentación y la gestión de peticiones.
- **Modelo.** Parte encargada de representar la información y las reglas asociadas a los datos. En una aplicación con persistencia, suele estar relacionado con las entidades, relaciones y operaciones sobre la base de datos.
- **Vista.** Parte responsable de presentar la información al usuario. En aplicaciones *web*, suele materializarse mediante plantillas HTML, hojas de estilo y contenido generado para el navegador.
- **Controlador.** Parte encargada de recibir las peticiones del usuario, coordinar la lógica necesaria y devolver una respuesta. Actúa como intermediario entre la vista, el modelo y otros servicios de la aplicación.
- **Patrón factoría.** Patrón creacional que centraliza la creación o selección de objetos, delegando en un componente especializado la decisión de qué implementación utilizar [16].
- **Modelo relacional.** Modelo de datos basado en tablas, filas, columnas, claves primarias, claves ajenas y restricciones de integridad.
- **Mapeo objeto-relacional.** Técnica que relaciona tablas de una base de datos con clases del lenguaje de programación, permitiendo trabajar con entidades en lugar de sentencias SQL directas.

Desarrollo web y seguridad

- **Arquitectura cliente-servidor.** Modelo en el que un cliente realiza peticiones y un servidor las procesa, ejecuta la lógica correspondiente y devuelve una respuesta.
- **Tokens JWT.** Mecanismo que representa información firmada sobre una identidad o sesión, permitiendo validar peticiones protegidas [12].

Visión por computador e inteligencia artificial

- **Detección de objetos.** Localiza entidades dentro de una imagen y les asigna una clase.

- **Estimación de puntos clave.** Localiza puntos relevantes de una estructura.
- **Segmentación de imágenes.** Divide una imagen en regiones o clases a nivel de píxel.
- **Clasificación de imágenes.** Asigna una o varias etiquetas a una imagen completa o a una región concreta.

Pruebas, calidad y despliegue

- **Pruebas unitarias.** Verifican componentes concretos de forma aislada.
- **Mocking.** Sustituye temporalmente una dependencia por una respuesta controlada, permitiendo aislar pruebas o simular errores.
- **Cobertura de código.** Métrica que indica qué partes del código han sido ejecutadas durante las pruebas.
- **Análisis estático.** Revisión del código fuente sin ejecutarlo para detectar problemas de mantenibilidad, duplicación, complejidad, seguridad o estilo.
- **Contenerización.** Técnica que empaqueta una aplicación junto con sus dependencias en una unidad reproducible.

4.2. Herramientas

Gestión, lenguaje y entorno

- **Jira.** Herramienta de gestión de proyectos [1].
- **Git.** Sistema de control de versiones [4].
- **GitHub.** Plataforma de alojamiento de repositorios Git [17].
- **Python.** Lenguaje de programación multiplataforma y multiparadigma, con un amplio ecosistema de bibliotecas para desarrollo *web*, procesamiento de datos, pruebas e inteligencia artificial [14].
- **Linux.** Familia de sistemas operativos libres.

- **Debian.** Distribución GNU/Linux caracterizada por su estabilidad [40].
- **Visual Studio Code.** Editor de código. [35].
- **DBeaver.** Herramienta gráfica de administración de bases de datos que permite inspeccionar esquemas, consultar tablas y ejecutar sentencias SQL [5].

Visión por computador e inteligencia artificial

- **YOLO.** Familia de modelos de detección de objetos cuyo nombre procede de *You Only Look Once*. Su propuesta original fue presentada por Redmon, Divvala, Girshick y Farhadi, y plantea la detección como un único problema de regresión en el que una red predice cajas delimitadoras y probabilidades de clase en una sola pasada [45].
- **YOLOv8.** Versión de la familia YOLO desarrollada por Ultralytics. Soporta tareas de visión por computador como detección de objetos, segmentación de instancias, clasificación de imágenes, estimación de pose y detección con cajas orientadas [53, 54].
- **Ultralytics.** Biblioteca de Python asociada a modelos YOLO modernos. Proporciona una interfaz para predicción, entrenamiento, validación, exportación y uso de modelos preentrenados [53].
- **MediaPipe.** *Framework* desarrollado por Google para construir soluciones de percepción mediante aprendizaje automático. Incluye tareas de visión como detección de manos, estimación de pose, análisis facial, segmentación, detección de objetos y reconocimiento de gestos [34, 9].
- **MediaPipe Tasks.** Interfaz de alto nivel de MediaPipe orientada a ejecutar tareas predefinidas de visión, audio y texto. En visión por computador incluye tareas como *Hand Landmarker*, *Pose Landmarker*, *Face Landmarker*, *Object Detector*, *Image Segmenter* y *Gesture Recognizer* [9].
- **Hand Landmarker.** Tarea de MediaPipe destinada a detectar puntos clave de las manos. Devuelve coordenadas de puntos de la mano, coordenadas del mundo y lateralidad de la mano detectada [10].
- **Pose Landmarker.** Tarea de MediaPipe destinada a detectar puntos clave del cuerpo humano en imágenes o vídeo. Devuelve puntos de

pose en coordenadas de imagen y coordenadas tridimensionales del mundo [11].

- **OpenCV.** Biblioteca de visión por computador y procesamiento de imágenes. Incluye funciones para lectura, escritura, transformación, filtrado y dibujo sobre imágenes [52].
- **NumPy.** Biblioteca de Python para cálculo numérico basada en arreglos multidimensionales y operaciones matriciales eficientes [8].

Desarrollo web, persistencia y configuración

- **Flask.** *Framework* de desarrollo *web* para Python. Permite definir rutas, procesar peticiones HTTP, devolver respuestas y estructurar aplicaciones mediante extensiones. También permite organizar rutas y vistas mediante *Blueprints*, un mecanismo de modularización propio de Flask. [41].
- **Jinja.** Motor de plantillas utilizado por Flask para generar HTML dinámico a partir de plantillas y datos del servidor [41].
- **HTML.** Lenguaje de marcado para páginas *web* [37].
- **CSS.** Lenguaje de estilos para la presentación visual de HTML [36].
- **JavaScript.** Lenguaje de programación ejecutado principalmente en el navegador, utilizado para añadir interacción y comunicación con servicios *web* [38].
- **Flask-JWT-Extended.** Extensión de Flask para crear, verificar y proteger rutas mediante *tokens* JWT [12].
- **Werkzeug.** Biblioteca usada por Flask. Incluye utilidades para peticiones, respuestas, seguridad, nombres de ficheros y contraseñas [42].
- **python-dotenv.** Biblioteca que permite cargar variables de entorno desde ficheros `.env` [31].
- **MariaDB.** Sistema gestor de bases de datos relacional, derivado de MySQL [13].
- **SQLAlchemy.** Biblioteca de Python para acceso a bases de datos y mapeo objeto-relacional. Permite definir modelos, relaciones y consultas desde el código [49].

- **Flask-SQLAlchemy.** Extensión que integra SQLAlchemy con Flask, simplifica la configuración de modelos y sesiones de base de datos [49].
- **SQLite.** Sistema gestor de bases de datos relacional ligero, embebido y basado en ficheros [50].
- **JSON.** Formato ligero de intercambio de datos basado en texto [26].

Pruebas, calidad, despliegue y documentación

- **Pytest.** *Framework* de pruebas para Python. Permite definir pruebas unitarias, utilizar *fixtures*, parametrizar casos y generar informes. [43].
- **unittest.mock.** Módulo de Python para crear objetos simulados y reemplazar dependencias durante una prueba [15].
- **GitHub Actions.** Plataforma de automatización de GitHub para definir flujos de integración continua, pruebas, análisis y despliegues [18].
- **SonarCloud.** Servicio de análisis estático de código orientado a detectar problemas de mantenibilidad, duplicación, seguridad, complejidad y calidad general [48].
- **Docker.** Plataforma para crear y ejecutar contenedores que empaquetan una aplicación con sus dependencias [25]. **Docker Compose.** Herramienta para definir y ejecutar aplicaciones formadas por varios contenedores relacionados mediante un archivo de configuración [24].
- **Kubernetes.** Plataforma de código abierto para la orquestación de contenedores. Permite automatizar el despliegue, escalado y gestión de aplicaciones contenedorizadas en entornos distribuidos [2].
- **dbdiagram.io.** Herramienta *web* para diseñar y representar modelos relacionales mediante una sintaxis textual [23].
- **Draw.io.** Herramienta para crear diagramas [28].
- **LaTeX.** Sistema de composición tipográfica utilizado en documentación científica y académica [6].
- **ChatGPT y Gemini.** Herramientas de inteligencia artificial generativa orientadas a la generación, revisión y transformación de documentación, código, pruebas, explicaciones e imágenes [7, 39].

5. Aspectos relevantes del desarrollo del proyecto

En este capítulo se recogen los aspectos más importantes del desarrollo del proyecto, desde la validación técnica inicial hasta la consolidación de una plataforma *web* modular y extensible. Se explican las decisiones que marcaron el diseño final, los problemas más relevantes encontrados durante el desarrollo y las soluciones aplicadas.

La aportación principal del proyecto no está únicamente en ejecutar modelos sobre imágenes, sino en haber construido una base capaz de organizar modelos, modos de ejecución, *pipelines*, permisos, resultados temporales y tareas de administración de forma coherente. Para explicar esta evolución, el desarrollo se resume en tres fases:

- **Validación técnica inicial.** Construcción de una demo para comprobar la ejecución de modelos de visión por computador desde una aplicación *web*.
- **Diseño de la aplicación escalable.** Definición de la arquitectura modular, el modelo de datos, la factoría de modelos, los *pipelines*, la configuración por etapas y la lógica de acceso.
- **Consolidación funcional.** Implementación de la administración de *pipelines*, resultados temporales, pruebas automatizadas, análisis estático, datos iniciales y despliegue reproducible.

5.1. Validación técnica inicial

El primer objetivo fue comprobar si el flujo básico de procesamiento era viable dentro de una aplicación *web*. Antes de diseñar usuarios, planes, permisos o administración, era necesario validar la parte de mayor riesgo técnico: recibir una imagen desde el navegador, procesarla en el servidor y devolver un resultado interpretable.

El flujo validado fue el siguiente:

1. Subida de una imagen desde el navegador.
2. Recepción y almacenamiento temporal en el servidor.
3. Selección del modelo y modo de procesamiento.
4. Ejecución del análisis en el *backend*.
5. Generación de una imagen procesada.
6. Devolución de una salida visual y estructurada.

La demo permitió comprobar dos tipos de análisis: detección de objetos mediante cajas delimitadoras y estimación de puntos clave mediante *landmarks*. La Figura 5.1 muestra una de las capturas de esta fase inicial, suficiente para representar el alcance de la demo: una imagen enviada desde la interfaz, procesada en el servidor y devuelta con anotaciones visuales.

Esta demo no se consideró un producto final. Su valor estuvo en reducir incertidumbre técnica y en ayudar al diseño de la aplicación final. A partir de ella se identificaron las partes que debían formalizarse: modelo de datos, permisos, factoría de modelos, *pipelines*, almacenamiento temporal, trazabilidad y administración.

5.2. Diseño de la aplicación escalable

La segunda fase consistió en transformar la demo en una plataforma extensible. El objetivo fue evitar una aplicación cerrada, con modelos y flujos fijados directamente en el código. Para ello, el diseño se apoyó en tres ideas: arquitectura modular, modelo de datos extensible y ejecución desacoplada de modelos.

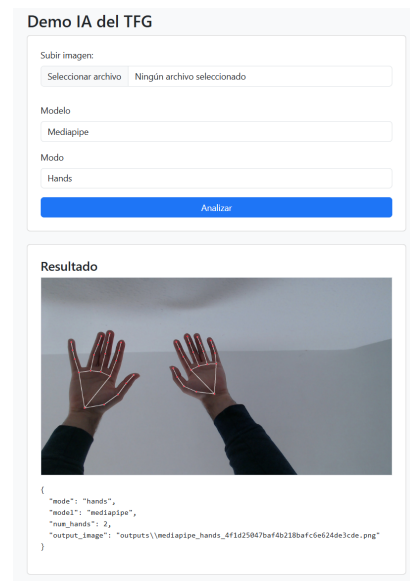


Figura 5.1: Resultado de la demo inicial con MediaPipe en modo manos, mostrando *landmarks* y conexiones sobre las manos detectadas.

Arquitectura modular

La arquitectura se organizó siguiendo una adaptación práctica del patrón Modelo-Vista-Controlador, incorporando una capa adicional de servicios. Esta capa resulta especialmente importante porque evita que los controladores acumulen lógica de procesamiento de imágenes, selección de modelos o ejecución de *pipelines*.

La Figura 5.2 representa esta organización general. La arquitectura no se limita al patrón clásico, sino que añade varios elementos propios del sistema:

- **Factoría de aplicación Flask.** Centraliza la creación de la aplicación Flask, la configuración y el registro de módulos, funciona mediante *blueprints* que separan rutas públicas, autenticación, tienda, *pipelines* y administración.
- **Capa de servicios.** Contiene la lógica de negocio, ejecución de *pipelines* y procesamiento de imágenes, en ella se encuentra una **factoría de inteligencia artificial**. Esta selecciona el motor de procesamiento adecuado sin acoplar los controladores a modelos concretos utilizando *Launchers* que encapsulan la ejecución específica de cada familia de modelos.

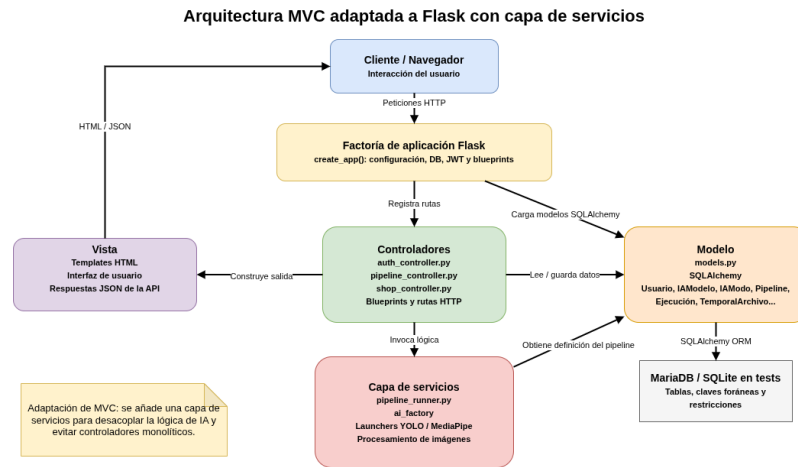


Figura 5.2: Organización arquitectónica inspirada en Modelo-Vista-Controlador, con una capa adicional de servicios para separar la lógica de procesamiento.

Esta separación permite que la lógica de entrada, la persistencia, la interfaz y la ejecución de modelos evolucionen de forma independiente.

Modelo de datos como base de la escalabilidad

El modelo de datos fue una de las partes clave del diseño, ya que permite escalar la aplicación sin depender de valores fijos en el código. La base de datos no solo representa usuarios, sino también roles, planes, suscripciones, alquileres de modelos, modelos de inteligencia artificial, modos de ejecución, *pipelines*, etapas, ejecuciones y archivos temporales.

La Figura 5.3 muestra el modelo entidad-relación del sistema. Su importancia está en que convierte los flujos de procesamiento en datos gestionables, no en funciones cerradas del código fuente.

La decisión más relevante fue separar los modelos de inteligencia artificial de sus modos de ejecución. De esta forma, un modelo puede disponer de varios modos, y cada etapa de un *pipeline* debe indicar tanto el modelo como el modo utilizado. Esto permite validar que una etapa no combine un modo con un modelo al que no pertenece.

También se separó la ejecución lógica de los archivos físicos generados. La entidad de ejecución conserva la trazabilidad del análisis, mientras que los archivos temporales mantienen rutas internas, *tokens* de descarga y fecha de

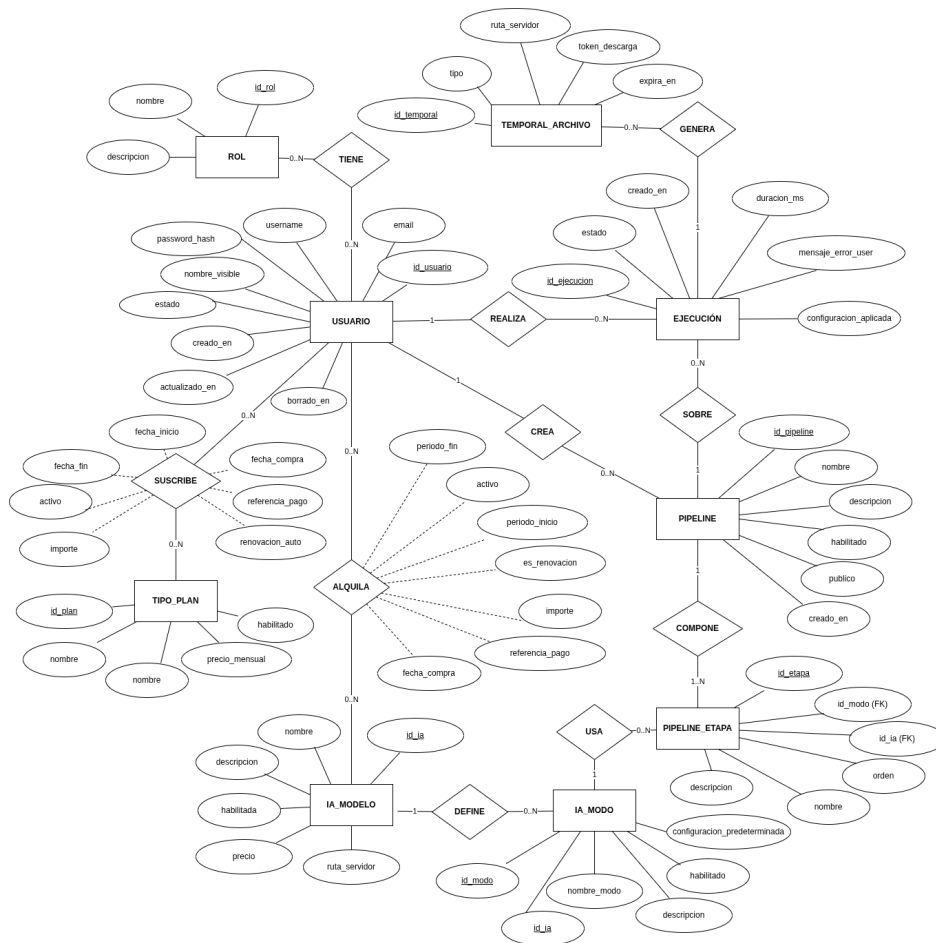


Figura 5.3: Modelo entidad-relación de la aplicación, utilizado como base para representar usuarios, permisos, modelos, modos, *pipelines*, ejecuciones y archivos temporales.

expiración. Así, el historial puede conservarse aunque los archivos generados ya no estén disponibles.

Pipelines de procesamiento

Los *pipelines* permiten pasar de una ejecución aislada de modelos a flujos formados por etapas. Cada etapa define qué modelo y qué modo deben ejecutarse, y en qué orden.

Esta estructura permite tres tipos de análisis:

- **Análisis simples.** Una única etapa, como detección de objetos, manos o pose.
- **Análisis compuestos.** Varias etapas encadenadas sobre una misma imagen.
- **Análisis ramificados.** Una etapa genera varias imágenes intermedias y la siguiente se aplica sobre cada una de ellas.

El caso más representativo de ejecución ramificada es el recorte de personas. Una etapa puede localizar varios sujetos y generar un archivo por cada uno. Después, una etapa posterior puede aplicar estimación de pose o detección de manos sobre cada recorte. Esto obligó a tratar las salidas como colecciones de archivos, no como una única imagen final.

Factoría de inteligencia artificial y launchers

La factoría de inteligencia artificial evita que el ejecutor de *pipelines* y los controladores dependan directamente de cada modelo concreto. Su objetivo es seleccionar el *launcher* adecuado según el modelo y el modo solicitados, manteniendo el sistema preparado para poder escalarlo.

La Figura 5.4 muestra el flujo de ejecución. El controlador valida la petición y delega en el ejecutor de *pipelines*. Este recupera las etapas desde la base de datos y, para cada una, solicita a la factoría el *launcher* correspondiente.

Esta decisión reduce el acoplamiento. Añadir una nueva familia de modelos no exige reescribir los controladores ni el ejecutor principal. El cambio se concentra en implementar un nuevo *launcher*, registrarlo en la factoría y asociar sus modos en la base de datos.

Configuración por etapas

Cada modo dispone de una configuración predeterminada en formato JSON. Estos valores actúan como plantilla inicial para ejecutar una etapa, pero pueden modificarse antes de lanzar el análisis.

Esta separación permite distinguir entre:

- **Configuración predeterminada.** Valores asociados al modo de ejecución.

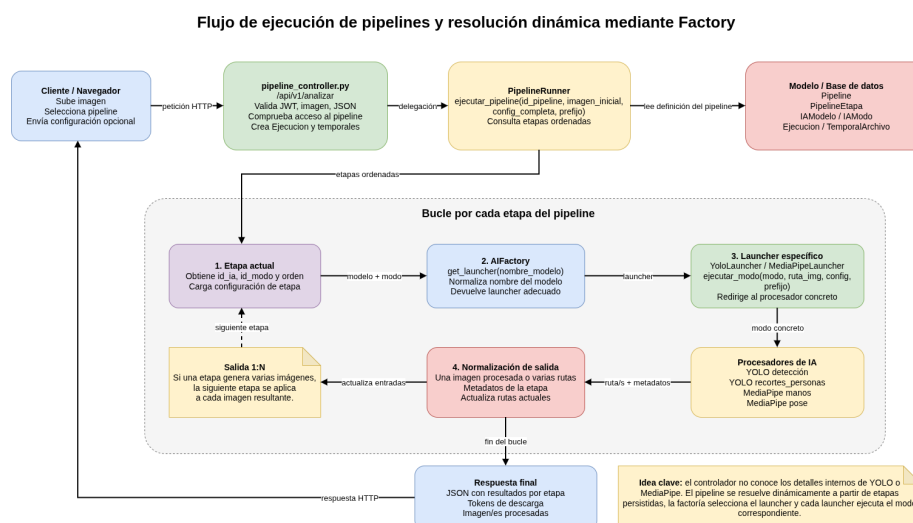


Figura 5.4: Flujo de ejecución de *pipelines* y resolución dinámica de modelos mediante la factoría de inteligencia artificial.

- **Configuración aplicada.** Valores concretos utilizados en una ejecución.

Gracias a ello, el comportamiento de una etapa puede ajustarse sin modificar la función de procesamiento. Por ejemplo, puede cambiarse un umbral de confianza o un parámetro de detección manteniendo el mismo *pipeline*.

Permisos y lógica de acceso

La aplicación incorpora autenticación mediante *tokens* JWT y autorización mediante roles, planes y licencias temporales. Esta lógica permite controlar qué usuarios pueden acceder a cada funcionalidad y qué *pipelines* tienen disponibles.

La plataforma diferencia entre:

- **Administradores.** Acceden a funcionalidades de gestión y revisión global.
- **Usuarios con plan Pro.** Acceden a todos los *pipelines* públicos y tienen una persistencia de archivos durante 30 días.

- **Usuarios básicos.** Acceden a *pipelines* según los modelos contratados y vigentes, y sus archivos se almacenan 5 minutos.

Esta lógica convierte la aplicación en algo más completo que una demo de inferencia. Los modelos y *pipelines* no solo existen en el sistema, sino que también están sujetos a reglas de acceso.

5.3. Consolidación funcional

La última fase consistió en completar la aplicación alrededor del diseño anterior: administración de *pipelines*, resultados temporales, pruebas automatizadas, análisis estático y preparación de un entorno reproducible.

Administración de pipelines sin modificar código

Una de las funcionalidades más importantes fue permitir la creación y edición de *pipelines* desde la zona de administración. La Figura 5.5 muestra esta parte de la interfaz, desde la que se pueden gestionar flujos formados por modelos y modos existentes.

Esta funcionalidad es relevante porque los *pipelines* dejan de ser una lista fija escrita en el código. El administrador puede definir etapas, ordenarlas, habilitar o deshabilitar flujos y modificar su disponibilidad sin cambiar la lógica principal de procesamiento.

Antes de guardar un *pipeline*, el *backend* valida que:

- El modelo exista y esté habilitado.
- El modo exista y esté habilitado.
- El modo pertenezca realmente al modelo indicado.
- Las etapas tengan una estructura válida.
- El *pipeline* pueda guardarse sin romper restricciones de base de datos.

Esta doble validación, desde aplicación y desde base de datos, refuerza la coherencia del sistema.

Pipelines
Gestiona flujos usando modelos y modos ya existentes. [Nuevo pipeline](#)

NOMBRE
Análisis Multi-Sujeto (Recorte + Pose)

VISIBILIDAD
☒ Público ☒ Activo

DESCRIPCIÓN
Aislamiento de individuos en imágenes separadas para analizar la pose de cada sujeto.

Etapas
Añade los modelos y modos en el orden de ejecución. [Añadir etapa](#)

Etapa 1 [Quitar](#)

NOMBRE
Aislamiento y Escalado

MODELO
yolo

MODO
recortes_personas

DESCRIPCIÓN
Segmentación de individuos y escalado de imágenes.

Etapa 2 [Quitar](#)

NOMBRE
Análisis de Pose Detallada

MODELO
MediaPipe

MODO
pose

DESCRIPCIÓN
Estimación esquelética aplicada a recortes de personas.

[Guardar](#) [Cancelar](#)

Figura 5.5: Interfaz de administración para crear y editar *pipelines* a partir de modelos y modos registrados.

Resultados temporales y ciclo de vida de los datos

El procesamiento de imágenes genera archivos que no deben exponerse mediante rutas internas del servidor. Para resolverlo, cada archivo generado se asocia a una ejecución y a un *token* de descarga.

Este diseño aporta varias ventajas:

- Oculta las rutas reales del servidor.
- Relaciona cada archivo con una ejecución concreta.
- Permite controlar la expiración de resultados.
- Mantiene trazabilidad aunque el archivo temporal desaparezca.

Además, se incorporaron tareas de mantenimiento para eliminar archivos expirados, renovar servicios cuando corresponde y anonimizar cuentas dadas de baja tras el periodo definido.

Reproducibilidad: datos iniciales y Docker

Para que la aplicación pueda probarse sin configurar manualmente todos los datos, se incorporó un *script* de inicialización. Este *script* prepara un entorno de demostración con usuarios, roles, planes, modelos de inteligencia artificial, modos y *pipelines* ya configurados.

Junto a los datos iniciales, se preparó un despliegue reproducible mediante Docker Compose. La aplicación se organiza en un contenedor para Flask y otro para MariaDB. De este modo, el entorno de ejecución queda definido de forma más controlada y se reducen errores derivados de diferencias entre equipos, versiones de dependencias o configuración del sistema operativo.

Pruebas automatizadas, errores corregidos y análisis estático

Las **pruebas automatizadas** se incorporaron para encontrar errores y comprobar que el sistema mantenía un comportamiento correcto. Se utilizó Pytest con una base de datos SQLite en memoria y técnicas de *mocking* para simular modelos, lectura de imágenes, escritura de archivos y fallos controlados.

La Tabla 5.1 resume los bloques cubiertos por las pruebas.

Bloque probado	Aspectos validados
Autenticación y usuarios	Registro, inicio de sesión, perfil, roles y baja lógica.
Tienda y acceso	Planes, alquileres, renovaciones y disponibilidad de modelos.
Pipelines	Listado, configuración, ejecución, historial y resultados.
Administración	Creación, edición, eliminación y validación de etapas.
Procesamiento	Imágenes, configuraciones, recortes, manos, pose y errores controlados.
Mantenimiento	Expiración de temporales, renovación de servicios y anonimización.

Tabla 5.1: Resumen de bloques cubiertos mediante pruebas automatizadas.

Estas pruebas ayudaron a detectar y corregir errores relacionados con entradas inválidas o nulas, configuraciones JSON mal formadas, datos fuera de rango, usuarios sin permisos, combinaciones incorrectas de modelo y modo, archivos expirados, archivos físicos inexistentes y fallos de escritura de imágenes.

Además, se incorporó **análisis estático** mediante SonarCloud integrado con GitHub Actions. Esta revisión permitió detectar mas problemas, especialmente incidencias de fiabilidad, mantenibilidad y seguridad.

Durante este proceso se corrigieron avisos relacionados con:

- **Fiabilidad:** tratamiento de errores y control de situaciones que podían provocar fallos no controlados.
- **Mantenibilidad:** simplificación de código, reducción de duplicidades y mejora de fragmentos señalados por la herramienta.
- **Seguridad:** revisión del 100 % de los *hotspots* detectados, especialmente los relacionados con el **tratamiento de *tokens*** y el envío de **cabeceras de autorización** desde la interfaz hacia rutas protegidas del *backend* además de corrección del **Dockerfile** para mejorar la seguridad y reproducibilidad de la imagen ya que se ajustó la instalación de dependencias para utilizar un entorno virtual interno, se incorporó un fichero **requirements.lock** con versiones fijadas y se revisaron los permisos de los directorios que requieren escritura, evitando depender innecesariamente del usuario *root*.

La Figura 5.6 muestra el resultado del análisis en SonarCloud, donde se alcanzó una valoración A en seguridad, fiabilidad y mantenibilidad, se revisó el 100 % de los *hotspots* de seguridad y se obtuvo una cobertura del 92,9 %.

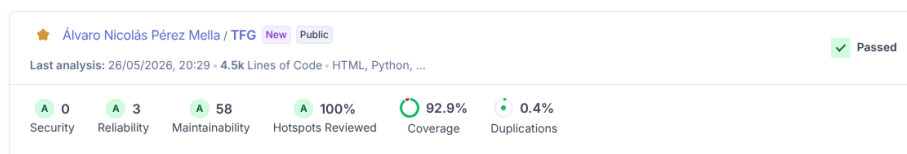


Figura 5.6: Informe de calidad del proyecto en SonarCloud.

La combinación de pruebas automatizadas, cobertura y análisis estático permitió validar tanto el comportamiento funcional del *backend* como aspectos internos de calidad del código antes de la entrega final.

6. Trabajos relacionados

Este capítulo relaciona el proyecto desarrollado con otras herramientas que también trabajan con modelos de inteligencia artificial y flujos de procesamiento de imágenes. El objetivo no es hacer un estudio exhaustivo de todas las soluciones existentes, sino mostrar de forma sencilla en qué se parece y en qué se diferencia el TFG respecto a algunas plataformas representativas.

6.1. Herramientas y plataformas relacionadas

Gradio es una biblioteca de Python orientada a crear interfaces *web* para modelos de aprendizaje automático de forma rápida [21]. Su finalidad principal es facilitar demos interactivas, con entradas y salidas visuales, sin construir una aplicación *web* completa desde cero.

Roboflow es una plataforma centrada en visión por computador. Ofrece herramientas para preparar conjuntos de datos, entrenar modelos, desplegarlos y construir *workflows* visuales mediante varios pasos de procesamiento [46]. Es la referencia más cercana al proyecto por la idea de combinar operaciones de visión por computador en un flujo.

Kubeflow Pipelines es una plataforma para definir y ejecutar flujos de aprendizaje automático formados por componentes encadenados [30]. Está orientada a entornos de mayor escala y a infraestructuras basadas en Kubernetes, por lo que su objetivo es más amplio que el de una aplicación *web* de análisis de imágenes.

6.2. Comparación con el proyecto desarrollado

La Tabla 6.1 resume las principales diferencias entre las soluciones revisadas y el TFG desarrollado.

Solución	Enfoque principal	Diferencia con el TFG
Gradio	Creación rápida de interfaces <i>web</i> para modelos.	El TFG no se limita a una demo; incorpora usuarios, permisos, persistencia, administración y resultados temporales.
Roboflow	Plataforma de visión por computador con <i>workflows</i> .	El TFG implementa una solución propia, con <i>pipelines</i> almacenados en base de datos y gestionables desde administración.
Kubeflow Pipelines	Orquestación de flujos de aprendizaje automático.	Kubeflow está orientado a Kubernetes; el TFG aplica la idea de ejecución por etapas a una plataforma específica de análisis de imágenes

Tabla 6.1: Comparación entre las soluciones relacionadas y el proyecto desarrollado.

Frente a Gradio, aporta una aplicación más completa, con autenticación, persistencia, permisos y administración. Frente a Roboflow, desarrolla una solución propia centrada en *pipelines* configurables dentro de la propia aplicación. Frente a Kubeflow Pipelines, comparte la idea de ejecución por etapas, pero la orienta a una plataforma *web* específica para análisis de imágenes.

7. Conclusiones y líneas de trabajo futuras

En este capítulo se presentan las principales conclusiones obtenidas tras la realización del trabajo, junto con las posibles líneas de evolución que permitirían continuar ampliando el proyecto en el futuro.

7.1. Conclusiones

La conclusión principal del trabajo es que el objetivo general del TFG se ha cumplido: se ha diseñado e implementado una plataforma *web* modular y extensible capaz de ejecutar flujos de procesamiento de imágenes mediante modelos de inteligencia artificial, devolviendo al usuario resultados visuales y datos estructurados.

El proyecto partió de una validación técnica inicial, centrada en comprobar si era posible subir una imagen desde el navegador, procesarla en el servidor y obtener una salida interpretable. A partir de esa demo, el desarrollo evolucionó hacia una aplicación completa con usuarios, permisos, *pipelines*, administración, resultados temporales, pruebas y despliegue reproducible. Esta evolución progresiva permitió reducir incertidumbre y construir el sistema de forma controlada.

En relación con los objetivos del *software*, se han implementado las funcionalidades previstas. La aplicación permite realizar el ciclo completo de uso: acceso del usuario, consulta de *pipelines*, subida de imágenes, ejecución del procesamiento, visualización de resultados y gestión administrativa. Por tanto, el sistema cubre las necesidades funcionales planteadas inicialmente.

Respecto a los objetivos técnicos, el resultado más importante ha sido la construcción de una arquitectura preparada para crecer. La separación entre interfaz, controladores, servicios, modelos de datos, factoría de inteligencia artificial y *launchers* evita una aplicación rígida y permite incorporar nuevos modelos, modos o *pipelines* con menor impacto sobre el flujo principal.

El modelo de datos y la gestión de resultados también se han diseñado con ese mismo enfoque. Los *pipelines* se representan como elementos configurables y no como funciones cerradas dentro del código. Además, los resultados generados se asocian a ejecuciones concretas y se gestionan de forma temporal, evitando exponer rutas internas del servidor.

La validación del sistema ha permitido comprobar y mejorar la calidad del desarrollo. Los tests unitarios automáticos del *backend* ayudaron a detectar errores relacionados con entradas inválidas, valores nulos, configuraciones incorrectas, permisos insuficientes, archivos expirados y fallos de escritura. Además, el análisis con SonarCloud permitió corregir problemas de fiabilidad, mantenibilidad y seguridad, alcanzando una valoración A en esas categorías y una cobertura final del 92,9 %.

La gestión del proyecto también ha sido clave para alcanzar el resultado final. La planificación por iteraciones permitió dividir el trabajo en fases revisables: primero la validación técnica, después el diseño escalable y, finalmente, la consolidación funcional, las pruebas, la documentación y el despliegue. Esta organización ayudó a priorizar los riesgos principales y a mantener una evolución constante del producto.

Desde el punto de vista personal, el trabajo ha permitido aplicar de forma integrada conocimientos del grado en desarrollo *web*, bases de datos, ingeniería del *software*, visión por computador, pruebas, seguridad y despliegue. También ha reforzado competencias de planificación, documentación y toma de decisiones técnicas.

En conjunto, el TFG ha alcanzado los objetivos planteados. El resultado no es solo una aplicación que ejecuta modelos de inteligencia artificial, sino una plataforma organizada para gestionar flujos de análisis visual. Su principal valor está en el diseño extensible: modelos, modos y *pipelines* quedan integrados en una arquitectura preparada para evolucionar sin rehacer el sistema desde cero.

7.2. Líneas de trabajo futuras

Aunque el proyecto cumple los objetivos definidos, existen varias líneas de mejora que permitirían continuar su evolución:

- **Integración de nuevos modelos y modos.** Ampliar el catálogo con nuevas familias de modelos de visión por computador, manteniendo la estructura basada en factoría y *launchers*.
- **Creación de pipelines por parte de usuarios.** Permitir que los usuarios creen sus propios *pipelines* desde la interfaz, combinando modelos y modos disponibles según sus permisos.
- **Compartición de pipelines con la comunidad.** Incorporar un sistema para publicar, reutilizar y valorar *pipelines* creados por otros usuarios, convirtiendo la plataforma en un entorno colaborativo.
- **Ejecución asíncrona de trabajos pesados.** Desacoplar las ejecuciones largas del ciclo directo de petición-respuesta, permitiendo procesar imágenes en segundo plano y consultar el estado del análisis.
- **Pasarela de pago real.** Sustituir la lógica simulada de planes y licencias por una integración real con un proveedor de pagos, manteniendo el control de acceso ya definido.
- **Mejora del panel de administración.** Ampliar las herramientas de administración con métricas de uso, filtros avanzados, revisión de errores y mayor control sobre modelos, modos y ejecuciones.
- **Refuerzo de seguridad y privacidad.** Profundizar en medidas de protección de datos, auditoría de accesos, gestión de sesiones y endurecimiento del despliegue.
- **Despliegue en entorno de producción.** Preparar una infraestructura con servidor *web* de producción, monitorización, copias de seguridad y configuración segura de servicios.

Bibliografía

- [1] Atlassian. Get started with jira. <https://www.atlassian.com/software/jira/guides/getting-started/introduction>, 2024. [Online; accedido 29-mayo-2026].
- [2] The Kubernetes Authors. Kubernetes overview. <https://kubernetes.io/docs/concepts/overview/>, 2026. [Online; accedido 30-mayo-2026].
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Software Freedom Conservancy. Git documentation. <https://git-scm.com/doc>, 2026. [Online; accedido 29-mayo-2026].
- [5] DBeaver Corporation. Dbeaver documentation. <https://dbeaver.com/docs/dbeaver/>, 2024. [Online; accedido 18-marzo-2026].
- [6] Colaboradores de Wikipedia. Latex. <https://es.wikipedia.org/w/index.php?title=LaTeX&oldid=84209252>, 2015. [Online; accedido 30-octubre-2025].
- [7] Google DeepMind. Gemini. <https://deepmind.google/models/gemini/>, 2024. [Online; accedido 29-mayo-2026].
- [8] NumPy Developers. Numpy documentation. <https://numpy.org/doc/stable/>, 2024. [Online; accedido 30-octubre-2025].
- [9] Google AI Edge. Mediapipe solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>, 2024. [Online; accedido 30-octubre-2025].

- [10] Google AI Edge. Hand landmarks detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker, 2026. [Online; accedido 29-mayo-2026].
- [11] Google AI Edge. Pose landmark detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/pose_landmarker, 2026. [Online; accedido 29-mayo-2026].
- [12] Flask-JWT-Extended. Flask-jwt-extended documentation. <https://flask-jwt-extended.readthedocs.io/en/stable/>, 2024. [Online; accedido 15-abril-2026].
- [13] MariaDB Foundation. Mariadb documentation. <https://mariadb.com/docs/>, 2024. [Online; accedido 18-marzo-2026].
- [14] Python Software Foundation. Python documentation. <https://docs.python.org/3/>, 2024. [Online; accedido 30-octubre-2025].
- [15] Python Software Foundation. unittest.mock — mock object library. <https://docs.python.org/3/library/unittest.mock.html>, 2024. [Online; accedido 14-mayo-2026].
- [16] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [17] GitHub. Github documentation. <https://docs.github.com/>, 2024. [Online; accedido 29-mayo-2026].
- [18] GitHub. Github actions documentation. <https://docs.github.com/en/actions>, 2026. [Online; accedido 27-mayo-2026].
- [19] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 4 edition, 2018.
- [20] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [21] Gradio. Gradio documentation. <https://www.gradio.app/docs>, 2026. [Online; accedido 30-mayo-2026].
- [22] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, 3 edition, 2011.
- [23] Holistics. dbdiagram.io documentation. <https://docs.dbdiagram.io/>, 2024. [Online; accedido 24-marzo-2026].

- [24] Docker Inc. Docker compose documentation. <https://docs.docker.com/compose/>, 2026. [Online; accedido 27-mayo-2026].
- [25] Docker Inc. Docker documentation. <https://docs.docker.com/>, 2026. [Online; accedido 27-mayo-2026].
- [26] ECMA International. The json data interchange syntax. <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>, 2017. [Online; accedido 24-marzo-2026].
- [27] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015.
- [28] JGraph. draw.io documentation. <https://www.drawio.com/doc/>, 2024. [Online; accedido 24-marzo-2026].
- [29] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413, 2019.
- [30] Kubeflow. Kubeflow pipelines documentation. <https://www.kubeflow.org/docs/components/pipelines/overview/>, 2026. [Online; accedido 30-mayo-2026].
- [31] Saurabh Kumar. python-dotenv. <https://pypi.org/project/python-dotenv/>, 2024. [Online; accedido 15-abril-2026].
- [32] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [33] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3431–3440, 2015.
- [34] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.

- [35] Microsoft. Visual studio code documentation. <https://code.visualstudio.com/docs>, 2024. [Online; accedido 29-mayo-2026].
- [36] Mozilla Developer Network. Css: Cascading style sheets. <https://developer.mozilla.org/en-US/docs/Web/CSS>, 2024. [Online; accedido 12-noviembre-2025].
- [37] Mozilla Developer Network. Html: Hypertext markup language. <https://developer.mozilla.org/en-US/docs/Web/HTML>, 2024. [Online; accedido 12-noviembre-2025].
- [38] Mozilla Developer Network. Javascript. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>, 2024. [Online; accedido 12-noviembre-2025].
- [39] OpenAI. Chatgpt. <https://openai.com/chatgpt>, 2024. [Online; accedido 29-mayo-2026].
- [40] Debian Project. Debian documentation. <https://www.debian.org/doc/>, 2024. [Online; accedido 18-marzo-2026].
- [41] Pallets Projects. Flask documentation. <https://flask.palletsprojects.com/en/stable/>, 2024. [Online; accedido 12-noviembre-2025].
- [42] Pallets Projects. Werkzeug documentation. <https://werkzeug.palletsprojects.com/en/stable/>, 2024. [Online; accedido 15-abril-2026].
- [43] pytest developers. pytest documentation. <https://docs.pytest.org/en/stable/>, 2024. [Online; accedido 14-mayo-2026].
- [44] Álvaro Pérez Mella. Picmind: Analizador de imágenes con flujos encadenados de inteligencia artificial. <https://github.com/alvaroooper/TFG.git>, 2026. Repositorio del proyecto. [Online; accedido 30-mayo-2026].
- [45] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [46] Roboflow. What are workflows? <https://docs.roboflow.com/workflows/what-is-workflows>, 2026. [Online; accedido 30-mayo-2026].

- [47] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2021.
- [48] SonarSource. Sonarqube cloud documentation. <https://docs.sonarsource.com/sonarqube-cloud>, 2026. [Online; accedido 27-mayo-2026].
- [49] SQLAlchemy. Sqlalchemy documentation. <https://docs.sqlalchemy.org/en/20/>, 2024. [Online; accedido 24-marzo-2026].
- [50] SQLite. Sqlite documentation. <https://www.sqlite.org/docs.html>, 2024. [Online; accedido 24-marzo-2026].
- [51] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.
- [52] OpenCV Team. Opencv library documentation. <https://docs.opencv.org/4.x/>, 2024. [Online; accedido 30-octubre-2025].
- [53] Ultralytics. YOLOv8. <https://docs.ultralytics.com/models/yolov8/>, 2024. [Online; accedido 30-octubre-2025].
- [54] Ultralytics. Computer vision tasks. <https://docs.ultralytics.com/tasks/>, 2026. [Online; accedido 29-mayo-2026].
- [55] Ian H. Witten, Eibe Frank, Mark A. Hall, and Christopher J. Pal. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, 4 edition, 2016.